

# 폐기 트리 샤딩 — 증명을 로그인당 1회로

PairCT Mode 2 폐기 구조 · 2026-09-04 설계, 2026-09-06 구현·  
전환 완료

**POSTECH**

POHANG UNIVERSITY OF SCIENCE AND TECHNOLOGY

# 지금 무엇이 문제인가

- **증명이 revocationRoot 하나에 통째로 묶여 있다**
  - pi\_pk\_i의 public signal에 revocationRoot가 들어간다
  - PPIDWallet.execute는 registry.isCurrentRoot(revocationRoot)를 먼저 본다
  - 따라서 폐기가 하나만 게시돼도 전 지갑의 증명이 동시에 죽는다
- **규모에서 이게 얼마나 나쁜가 (100M DAU, 폐기 1만/일 = 0.01%/일 가정)**
  - 폐기 0.116건/초 = 블록당 약 1.4건 — 매 블록 root가 바뀐다
  - 증명 생성에 755.8ms가 걸리는데, 만들자마자 죽는다
- **지금은 배칭이 이걸 가려주고 있다**
  - 5분 배칭이면 폐기 35건을 root 변경 1회로 묶으므로 증명이 5분 산다
  - 대신 폐기가 최대 5분 늦게 듣는다 — BAAR 대비 우리가 지는 유일한 지점
  - **배칭 주기 = 증명 수명 = 폐기 지연이 한 덩어리로 묶여 있다**

# 먼저: root와 무관한 증명은 만들 수 없다

- 두 요구가 동시에 걸려 있다
  - 리프(L\_acct, L\_sess)는 숨겨야 한다 — 드러나면 교차-RP 연결이 뚫린다
  - 컨트랙트는 현재 폐기 상태에 대해 검증해야 한다
- 그런데 컨트랙트는 숨겨진 값의 멤버십을 볼 수 없다
  - 따라서 신선도 검사는 회로 안에 있어야 하고, 그러면 회로가 root에 묶인다
  - 커밋먼트로 감싸도 마찬가지다 — 컨트랙트는 여전히 그 값을 트리에서 못 찾는다
- 그래서 레버는 셋뿐이다
  - ① 어떤 root에 묶느냐 — root가 덜 자주 바뀌게 만든다 (4~10번)
  - ② 그 root가 얼마나 오래 유효하냐 — grace window (11번)
  - ③ 그 root를 누가 정하느냐 — 갱신 정당성 (12~13번)

# 설계: 깊이 20 트리를 중간에서 자른다

latestTopRoot (bytes32 1개) · 레지스트리는 지금 그대로

상위 트리 · keccak256 — 컨트랙트가 Solidity에서 검증

절단면

세션 서브트리 4,096개  
Poseidon 깊이 8

계정 서브트리 1,024개  
Poseidon 깊이 10

회로가 비멤버십을 증명하는 구간 — SNARK는 sess\_root와 acct\_root에만 묶인다

상위 트리는 회로에 들어가지 않으므로 SNARK 친화적일 필요가 없다 — 그래서 keccak을 쓴다. 회로 안의 keccak은 해시당 15만 constraint급이라 불가능하다.  
총 경로 길이는 보존된다: 세션 8+12, 계정 10+10. 자르는 위치만 바뀔 뿐 용량도 그대로다.  
두 층의 샤딩 기준이 다르다(6~7번). 세션은 만료 기반, 계정은 값 기반이다.

# 그 결과: 증명이 두 조각으로 나뉜다

|          | 담는 것                                       | 언제 죽나                        | 비용            |
|----------|--------------------------------------------|------------------------------|---------------|
| SNARK    | target $\notin$ 내 서브트리<br>(세션·계정 각 1건)     | 내 서브트리가 바뀔 때만                | 510.3 ms (실측) |
| calldata | sess_root, acct_root,<br>샤드 인덱스, 상위 형제 22개 | top root가 바뀔 때<br>(= 아무 폐기나) | 무료 · 제출 직전 갱신 |

컨트랙트가 (root\_s, 샤드 인덱스, 형제)로 top root를 재계산해 latestTopRoot와 비교한다.

**핵심: 다른 서브트리에서 폐기가 나도 내 root\_s는 그대로다 → 비싼 SNARK가 살아남는다.**

바뀌는 것은 상위 형제뿐이고, 그건 루트 목록만 다시 받으면 되는 calldata다.

실측: 샤드 인덱스 제약은 0 constraint다 — 회로가 이미 만든 targetBits의 선형 결합이라 공짜다.

# 세션과 계정은 샤딩 기준이 다르다

| 층  | 샤딩 기준                                                        | 증명자가 자기 샤드를 아는가                         | 회수                            |
|----|--------------------------------------------------------------|-----------------------------------------|-------------------------------|
| 세션 | $(\text{max\_height} \bmod 512) \times 8$<br>+ L_sess 하위 3비트 | 안다 — max_height가 이미 public signal이다     | 공짜: 만료 블록이 지나면<br>샤드를 통째로 리셋  |
| 계정 | L_acct 하위 10비트                                               | 유도한다 — 회로가 $s == f(\text{target})$ 을 강제 | 재기준화. 단 샤드 단위라<br>그 샤드 지갑만 영향 |

세션에서 만료 기반 샤딩이 성립하는 이유: 세션 리프의 만료가 곧 max\_height이고, 그건 이미 공개값이다.

assertMaxHeightWithinBound가 max\_height <= 현재 + 300 + 32를 보장하므로 512칸 링이면 충분하다.

**결과가 크다: 세션 층에서 재기준화·epoch·용량 정책이 통째로 사라진다. 만료된 샤드는 빈 트리 상수로 리셋하면 끝이다.**

계정은 안 된다 — 계정 폐기의 만료는 접수 시점 + 332라 증명자가 모른다(모르는 것이 요점이다).

# 리뷰에서 발견: 세션 층이 병목이었다

- 빠뜨린 것 두 가지

- **SNARK는 sess\_root와 acct\_root를 둘 다 public으로 갖는다** — 둘 중 먼저 바뀌는 쪽이 증명을 죽인다
- 세션 샤드를  $\text{max\_height} \bmod 512$ 로만 잡으면 칸은 512개지만 실효는 332개다 ( $\text{max\_height}$  범위가 332 블록이라 그 이상 헤어지지 않는다)

- **그래서 결합해 다시 계산하면 (폐기 1.4건/블록)**

- 폐기가 전부 세션이면 239블록(48분)마다 변경 → 크레덴셜 1시간당 1.26회 재생성
- **계정 층만 세면 0.41회지만 그건 세션 층을 빼놓은 값이다** — 초안이 그렇게 낙관적으로 잡혀 있었다

- **해법: 세션 샤드를 2차원으로**

- 샤드 =  $(\text{max\_height} \bmod 512) \times 8 + L\_sess$  하위 3비트 → 4,096칸(실효 2,656)
- 상위는 컨트랙트가  $\text{max\_height} \% 512$ 로 직접 계산하고, 하위 3비트만 회로가 고정한다 — 회로에 나눗셈이 필요 없다
- **결과: 재생성 0.16~0.41회. 그리고 세션 깊이가 8로 줄어 회로가 더 싸진다(13,905 실측)**

# 효과: 시간당 12회 → 로그인당 1회

| 폐기 구성 (총 1.4건/블록) | 내 서브트리 변경 주기    | 크레덴셜 1시간당 SNARK 재생성 |
|-------------------|-----------------|---------------------|
| 전부 세션 폐기          | 1,911블록 (6.4시간) | 0.16회               |
| 세션·계정 절반씩         | 1,063블록 (3.5시간) | 0.28회               |
| 전부 계정 폐기          | 737블록 (2.5시간)   | 0.41회               |
| 지금 (배칭 5분, 샤딩 없음) | 25블록 (5분)       | 12회                 |

어느 구성이든 변경 주기가 크레덴셜 수명(300블록 = 1시간)보다 길다 → 로그인 1회에 증명 1개가 보통이다.  
증명이 짧게 사는 원인은 SNARK가 무거워서가 아니라, 리프 100만 개를 root 하나로 묶어놔기 때문이다.  
묶음을 나누면 무효화 빈도도 그만큼 나뉜다. 남은 병목은 계정 총(0.41회)이고, 계정 샤드를 4,096으로 올리면 더 내려간다.

# 이 설계의 유일한 민감 변수: 폐기율

| 시나리오               | 폐기/일   | 폐기/블록  | 상주 리프 | 크레덴셜 1시간당<br>SNARK 재생성 (절반씩 기준) |
|--------------------|--------|--------|-------|---------------------------------|
| 0.001%/일 (계정 사고만)  | 1,000  | 0.14   | 42    | 0.03회                           |
| 0.01%/일 (이 텍스트 기준) | 1만     | 1.4    | 417   | 0.28회                           |
| 0.1%/일             | 10만    | 13.9   | 4,167 | 2.8회                            |
| 로그인의 1%            | 100만   | 139    | 4.2만  | 28회                             |
| 로그인의 10%           | 1,000만 | 1,389  | 42만   | 282회                            |
| 로그아웃마다 (100%)      | 1억     | 13,889 | 417만  | 2,819회                          |

마지막 줄: 로그아웃마다 폐기하면 상주 리프가 417만 — 멤버십 트리와 정확히 같아지고 비멤버십의 이점이 사라진다.

**정책으로 못박아야 한다 — 로그아웃은 폐기하지 않는다. 1시간 자연 만료로 충분하고, 폐기는 보안 사건에만 쓴다.**

샤드 개수는 거의 공짜 knob이다: 회로 비용은 서브트리 깊이만 따르고 온체인은  $\log_2(\text{샤드}) \times \text{keccak}(\text{레벨당 약 } 1,100 \text{ gas})$ 이다.

그래서 폐기율이 높으면 샤드를 늘리면 된다 — 0.1%/일 구간도 샤드를 16배로 키우면 크레덴셜 수명을 넘긴다. 회로는 그대로다.

# 비용 — 회로는 오히려 싸진다 (constraint는 실측)

| 항목            | 지금            | 제안                        | 출처 |
|---------------|---------------|---------------------------|----|
| 세션 비멤버십       | 6,648 (깊이 20) | 3,732 (깊이 8)              | 실측 |
| 계정 비멤버십       | 6,648 (깊이 20) | 4,218 (깊이 10)             | 실측 |
| 전체 회로         | 19,251        | 13,905 (-27.8%)           | 실측 |
| 증명 시간         | 755.8 ms      | 510.3 ms                  | 실측 |
| execute 가스    | 276,709       | 330,079 (+53,370, +19.3%) | 실측 |
| public signal | 6개            | 9개                        | 실측 |
| 온체인 상태        | bytes32 1개    | 동일                        |    |

circom 2.1.9 단독 컴파일. build/는 건드리지 않았다. results/mode2\_sharded\_circuit\_constraints\_20260904.csv

기준선 재현 확인: IMTNonMembershipV2(20) = 6,648, 현행 pi\_pk\_i = 19,251 — 기록값과 일치.

**레벨당 243, 고정 오버헤드 1,788로 분해된다. 상위 22단을 Poseidon에서 keccak으로 옮긴 만큼이 그대로 이득이다.**

**가스가 추정(+12,000)을 4배 넘었다. 상위 경로 keccak과 calldata만 세고, Groth16 verifier가 public signal마다 치르는 비용(6->9개, ecMul 3회 약 18k)을 빼먹었다.**

증명 시간은 세 회로를 한 실행에서 순차로 돌린 통합 벤치 기준이다(데모 스택 정지). 로그인 1회분 합은 912.7 ms.

# 함께 엮을 것: 블록 단위 grace window

- 왜 필요한가

- top root는 여전히 매 블록 바뀌므로, 제출과 채굴 사이에 바뀌면 revert한다
- 복구 비용은 가벼워졌지만(형제만 갱신, SNARK 재생성 불필요) 아예 없애는 편이 낫다

- 방식

- 레지스트리가 (root, 게시블록)을 저장하고, 최근 K블록 안에 게시된 root를 허용한다
- 폐기 지연 = K블록로 고정된다 — 폐기 빈도와 무관하다
- $K = 2 \sim 3$ 이면 24~36초. 지금 배칭 지연(수 분)보다 오히려 짧다

- 설계 주의

- **grace**는 **root** 개수가 아니라 블록 수로 재야 한다 — 개수로 재면 변경이 드문 샤드에서 옛 **root**가 오래 남는다
- 이걸 43ca1bc(grace 제거)를 의식적으로 되돌리는 것이다. 당시 근거는 '배칭 덕에 root가 드물게 바뀐다'였고, 그 전제가 바뀐다

# 지금 누가 트리를 관리하고 누가 게시하나

| 역할                     | 주체                                 | 자격                                |
|------------------------|------------------------------------|-----------------------------------|
| 폐기 접수 · 트리 보관 · 상태 영속화 | IdP 프로세스 (custom_idp.js)           | — (체인 쓰기 없음, eth_blockNumber 읽기만) |
| 폐기 요청 · 계정 차단 · 재기존화   | 운영자                                | IDP_ADMIN_SECRET                  |
| 온체인 pushRoot           | 운영자 (sweep 데몬)                     | REVOCATION_IDP_ADDRESS의 이더리움 키    |
| 게시 승인                  | RevocationRegistry.idp (immutable) | onlyIdP                           |

IdP에 개인키를 두지 않으려고 게시를 분리했는데, sweep 데몬이 두 자격을 다 들고 있어 분리가 명목상이다.

**컨트랙트는 게시된 root가 실제 트리와 맞는지 전혀 검증하지 않는다 — 갱신 정당성 증명이 없다.**

그래서 레지스트리 키를 가진 사람은 (a) 옛 root를 다시 올려 전원의 폐기를 되돌리거나 (b) 쓰레기 root로 전원을 막을 수 있다.

사용자는 자기 세션을 폐기할 수 없다 — /idp/revoke는 관리자 전용이다.

### ③ 컨트랙트가 트리를 유지하게 하면?

- 안 되는 두 가지부터

- 트리 전체 저장: 리프 100만 개 — 스토리지에서 끝난다
- 삽입마다 **root** 재계산: **Poseidon** 약 40회. **EVM에 precompile이 없어 폐기 1건당 100만 gas 안팎이다**
- Poseidon을 keccak으로 못 바꾼다 — 같은 트리를 회로에서도 봐야 하는데 회로 안 keccak은 해시당 15만 constraint급이다

- **실현 가능한 형태: 갱신 정당성 증명 (zk-rollup 방식)**

- `pushRoot(oldRoot, newRoot, proof)` — 컨트랙트가 `oldRoot == latestRoot`를 강제하고 전이 증명을 검증한다
- Groth16 1회 약 25만 gas인데 게시 배치당 한 번이다 — 배치 35건이면 폐기 건당 7,000 gas

- 무엇이 막히고 무엇은 안 막히나

- 막힘: 옛 **root** 재게시(폐기 되돌리기), 쓰레기 **root**로 전원 차단
- 안 막힘: 폐기 누락. 체인은 무엇이 폐기돼야 하는지 모른다 — 어떤 온체인 장치도 이걸 못 막는다
- 세션 층은 증명이 아예 필요 없다 — 샤드 인덱스가 곧 만료라 컨트랙트가 `block.number`만 보고 스스로 리셋한다

# 함께 해결되는 기존 약점

| 기존 약점                        | 해결                             |
|------------------------------|--------------------------------|
| 폐기 지연이 게시 주기만큼 (수 분)         | K블록으로 고정 — 24~36초              |
| root 전환 시 인플라이트 revert       | grace가 흡수                      |
| 재기준화가 전 지갑에 $O(n)$ 재다운로드를 물림 | 샤드 단위 — 해당 샤드 지갑만              |
| 세션 폐기도 재기준화로만 회수됨            | 세션 층은 재기준화 자체가 사라짐 — 만료 샤드를 리셋 |
| 지갑 콜드 스타트가 $O(n)$            | 리프 $n$ /샤드 + 루트 목록             |
| 증명이 매 게시마다 죽음                | 내 서브트리 변경 시에만                  |
| 운영자가 옛 root를 올려 폐기를 되돌릴 수 있음 | 갱신 정당성 증명으로 차단 (13번)           |

첫 줄이 BAAR 대비 우리가 지는 유일한 지점이었다(세미나 텍 26번). 이 설계가 그걸 없앤다.

마지막 줄은 BAAR가 Lemma 6에서 주장하지만 증명하지 못하는 지점이다 — 우리는 실제로 채울 수 있다.

# 반드시 지켜야 할 소트니스 조건

- 계정 총

- ① 회로가  $s == f(L\_acct)$ 을 강제해야 한다 — 없으면 증명자가 비어 있는 서브트리를 골라 통과한다

- 세션 총 — 상위·하위를 서로 다른 주체가 고정한다

- ② 컨트랙트가 상위를  $\max\_height \% 512$ 로 직접 계산해야 한다 (호출자가 준 값을 쓰면 안 된다)
- ③ 회로가 하위 3비트  $== L\_sess$  하위 3비트를 강제해야 한다
- 둘 중 하나만 있으면 나머지 축이 자유로워져 빈 샤드를 고를 수 있다

- 양쪽 공통

- ④ IdP는 반드시  $f(leaf)$  샤드에만 삽입해야 한다 — 엉뚱한 샤드에 넣으면 그 폐기는 증명에 보이지 않아 조용히 무효가 된다
- 지금 구조에 없던 종류의 실패 모드다. 테스트로 고정해야 한다

- 함께 검증할 것

- 부하 균형은 샤드별로 봐야 한다 — 전역 용량 정책 1개가 샤드 수만큼으로 늘어난다
- 재기준화가 샤드 단위가 되면서 epoch도 샤드별이 된다 — 지갑 동기화 프로토콜 변경

# 검토했다가 버린 안

| 안                                       | 왜 버렸나                                                                                                              |
|-----------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| 멤버십으로 전환 (BAAR 방식)                      | 트리 크기가 로그인 빈도에 비례 → 100M DAU에서 상주 리프 417만, 초당 2,315 쓰기.<br>지갑이 코어 27%를 상시 태워야 한다. 그리고 배칭 지연이 폐기가 아니라 매 로그인으로 옮겨간다  |
| L_sess를 공개해 세션 비멤버십을<br>온체인 keccak으로 검증 | 회로가 크게 싸지지만, ldp가 issuanceLog를 r_token으로 키잉해 갖고 있다.<br>체인만 보면 uid를 얻는다 → 조건부 추적이 상시 추적이 된다                         |
| 만료 기반 샤딩 — 계정 층에 한해                     | 계정 폐기의 만료는 접수 시점 + 332라 증명자가 모른다 → 전 샤드 증명이 필요하다.<br>세션 층에서는 만료가 곧 public signal인 max_height라 성립하며, 채택했다(6번)       |
| 상위 경로도 회로 안에 두기                         | 총 경로 길이가 보존되므로 이득이 0이다. 밖으로 빼야 의미가 생긴다                                                                             |
| 서브트리 root를 스토리지에 직접 저장<br>(top 트리 없이)   | 사용자 가스는 오히려 싸다(SLOAD 2회 vs keccak 22회). 보류 사유는 확장성이다 —<br>샤드를 $2^{16}$ 이상으로 늘리는 경로에서 mapping 슬롯 비용이 붙는다. 재검토 여지 있음 |

# 남은 위험과 전제

- 가스 — 배포 체인 선택에 의존한다

- 매 블록 게시는 하루 7,200 tx, 약 1억 8,700만 gas
- 메인넷 **20 gwei**면 하루 약 **3.75 ETH** — 비현실적. L2 또는 전용 체인 전제다

- 무엇이 실측이고 무엇이 아닌가

- constraint(3,732 / 4,218 / 13,905)는 circom 단독 컴파일 실측이다 — 기준선 6,648·19,251 재현으로 검증했다
- 증명 시간(**510.3 ms**)과 가스(**330,079**)도 이제 실측이다 — **zkey**를 만들고 진짜 증명으로 **execute**를 통과시켰다
- 추정으로 남은 것은 온체인 Poseidon 100만 gas 하나다 — 이 설계는 keccak만 쓰므로 직접 영향은 없다

- 가정에 의존한다

- 폐기 블록당 **1.4건(1만/일, 0.01%)**을 전제로 한 결론이다 — **9번의 민감도 표 참조**
- 세션·계정 폐기 비율은 모른다. 8번은 세 경우를 모두 제시했다

- 구현 규모와 재배포

- lib/, IdP의 v2 섹션 전체, 지갑 동기화, 회로, 상태 파일 v5 + 마이그레이션
- zkey 재생성 + PPIDWalletFactory 재배포. chain\_id를 public signal로 올리는 수정과 묶는 것이 낫다

# 진행 상황과 남은 것

- 완료 (2026-09-04 ~ 09-06)

- 설계 문서, 회로(pi\_pk\_i\_v3), 라이브러리(imt\_v3), 컨트랙트(RegistryV3/WalletV3), IdP·지갑 배선
- zkey 생성 → 진짜 증명으로 execute 통과 → 실운영 스택 전환까지 마쳤다
- 실측 확정: **constraint 13,905 · 증명 510.3 ms · 가스 330,079**
- 테스트 unit+circuit 16 / chain 3 / contract 31 / live 16 전부 통과

- 논문 (final47에 반영 완료)

- 폐기 절(IV-G)과 Property 9 신설, Table 1에 Revocation 열과 BAAR 행
- 정정: "새 신뢰 주체 없음"은 거짓이 됐다 — 온체인 게시자가 상시 역할이다
- Property 5를 off-chain transcript로 한정 (계정 샤드 인덱스 누출)

- 남은 것

- v2 경로 정리(회로·IdP 트리·옛 컨트랙트)는 안정화 후
- 갱신 정당성 증명(12번)은 별도 설계 — 롤백은 막지만 폐기 누락은 원리상 못 막는다
- 온체인 Poseidon 가스만 추정으로 남았다(이 설계는 keccak만 쓴다)